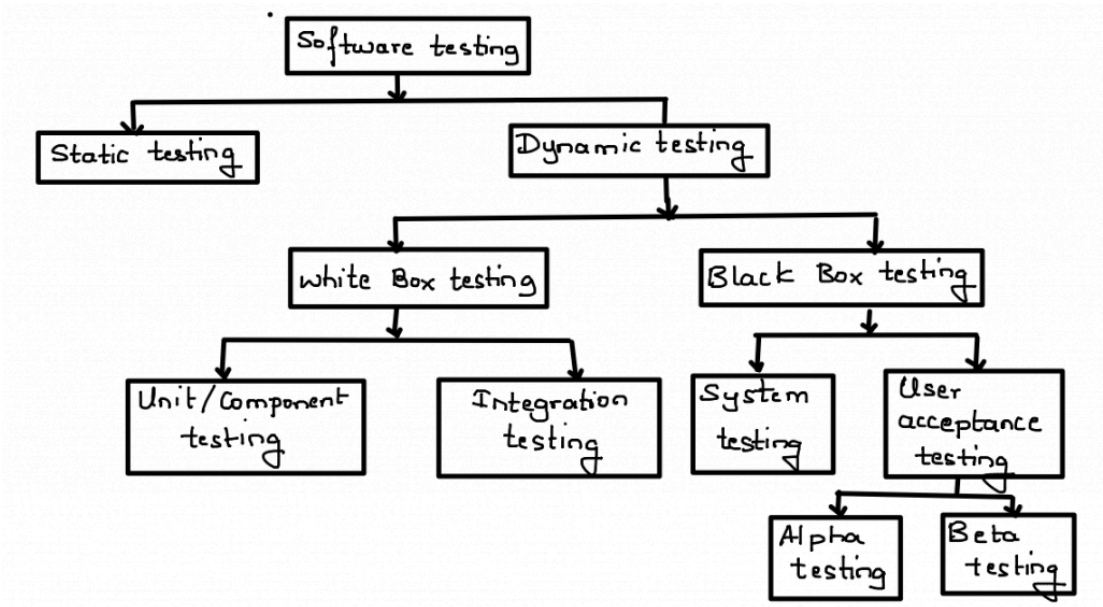
Laborator 05 - Unit Testing

Software testing

Clasificare

Tipurile de testare software pot fi clasificate în linii mari ca testare statică și testare dinamică. În ceea ce privește testarea ***statică***, documentele proiectului software sunt revizuite pentru a identifica erorile. În testarea ***dinamică***, software-ul este testat în timpul execuției. Testarea dinamică poate fi white-box testing (am acces la cod - testez o zona specifică) sau black-box testing (nu am acces la cod - testez output general pentru un input specific).



Cele două tipuri de testare ***white-box*** sunt ***testarea unitară*** și ***testarea de integrare***. Testarea white-box este denumită și ***testare structurală***. ***Testarea unitară*** sau testarea componentelor testează fiecare componentă a programului în mod izolat. ***Testarea de integrare*** reprezintă un mod de a testa interfețele dintre componentele programului - a testa modul în care anumite feature-uri interacționează între ele.

Cel de-al doilea tip de testare dinamică este testarea ***black-box*** - numită si testare bazată pe ***specificații***. ***Testarea sistemului*** și ***testarea de acceptanță a utilizatorului*** (UAT) sunt cele două moduri de testare black-box. ***Testarea sistemului*** se bazează pe testarea întregului sistemul si a modului său de funcționare. ***Testarea de acceptanță*** determină în ce măsură o aplicație îndeplinește nevoile utilizatorilor finali.

Unit testing

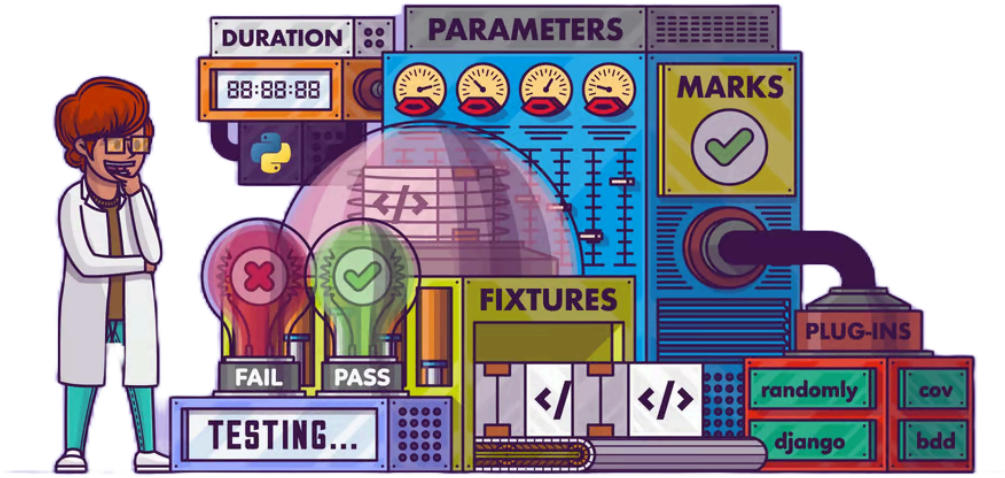
Un test unitar reprezintă o modalitate de a ***testa o unitate*** - cea mai mică bucată de cod care poate fi izolată logic dintr-un sistem. În majoritatea limbajelor de programare, aceasta este o funcție, o subrutină, o metodă sau o proprietate. În ceea ce privelte paradigma orientată pe obiecte, unitatea care urmează să fie supusă testării este adesea o clasă, o interfață sau o metodă.

Testarea unitară este un pas important în procesul de dezvoltare, deoarece, dacă este făcută corect, poate ajuta la ***detectarea din timp a bug-urilor***, care pot fi mai dificil de găsit în etapele ulterioare de testare.

Testarea unitară este o componentă a ***dezvoltării bazate pe testare (TDD)*** - o abordare prin dezvoltarea unui produs prin intermediul testării și revizuirii continue. Această metodă de testare este, de asemenea, primul nivel de testare a software-ului, care este efectuat înaintea altor metode de testare, cum ar fi testarea de integrare. Testele unitare sunt de obicei izolate pentru a se asigura că o unitate nu se bazează pe altă bucată de cod sau funcții externe.

pytest

PyTest este un framework de testare care permite utilizatorilor să scrie teste folosind limbajul de programare Python. Oferă o funcționalitate completă care permite scrierea de teste unitare simple până la teste funcționale complexe.



Pytest spre deosebire de bilioteca default de testare oferită de Python - unittest [https://docs.python.org/3/library/unittest.html] - oferă o flexibilitate și o putere mai mare de testare.

Documentația oficială pytest: https://docs.pytest.org/en/7.2.x/ [https://docs.pytest.org/en/7.2.x/]

Set-up

Instalare python

Pentru Ubuntu:

```
$ sudo apt install python3
```

Pentru Windows:

https://www.python.org/downloads/ [https://www.python.org/downloads/]

Asigurați-vă ca aveti versiunea de ***python3.7+***

```
$ python3 --version
```

Instalare pytest

Pentru Ubuntu:

```
$ sudo apt install python3-pip

$ pip3 install pytest
```

Pentru Windows:

Deschide o instanță de ***cmd***

```
$ pip3 install pytest
```

Creare primului test

Pentru ca pytest să recunoască un fișier de test, acesta trebuie să conțină prefixul **test_** (e.g. test_factorial.py). De asemenea, și metoda trebuie să conțină prefixul **test_** (e.g. def test_i_love_tsc()).

```
# content of test_TSC.py

def increment(x):
    return x + 1

def test_increment():
    assert increment(3) == 4
```

Pentru a rula pytest pentru fișierul anterior:

```
$ pytest test_TSC.py
```

În cazul în care aveți mai multe fișiere de test și doriți execuția tuturor testelor, se execută doar comanda de pytest.

```
$ pytest
```

În cazul în care doriți să rulați un test specific, folosiți

```
$ pytest $NUME_FISIER.py::$NUME_TEST

$ pytest test_TESC.py::test_increment
```

Un flag important pentru pytest este -k care primește ca input un string și rulează doar testele care se potrivesc cu secvența string.

```
$ pytest -k <substring>

$ pytest -k increment
```

Exemplu output test - passed

```
============================================= test session starts ==============================================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: $PATH
collected 1 item

test_TSC.py .                                                                                          [100%]

============================================== 1 passed in 0.09s ===============================================
```

Exemplu output test - failed

```
============================================= test session starts ==============================================
platform linux -- Python 3.10.6, pytest-7.2.2, pluggy-1.0.0
rootdir: $PATH
collected 1 item

test_TSC.py F                                                                                          [100%]

=================================================== FAILURES ===================================================
_____________________________________________ test_increment _____________________________________________
    def test_increment():
>       assert increment(3) == 5
E       assert 4 == 5
E        +  where 4 = increment(3)

test_TSC.py:5: AssertionError
=========================================== short test summary info ============================================
FAILED test_TSC.py::test_increment - assert 4 == 5
============================================== 1 failed in 0.15s ===============================================
```

Assert statements

1. Equal to or not equal

```
assert 5 == 5 || assert 5 != 3 # Success Examples
assert 5 == 3 || assert 5 != 5 # Fail Examples
```

2. type()

```
assert type(5) is int # Success Example
assert type(5) is not int # Fail Example
```

3. isinstance

```
assert isinstance('5', str) # Success Example
assert isinstance('5', int) # Fail Example
```

4. Boolean

```
truth_variable = 5 == 5
assert truth_variable is True # Success Example
assert truth_variable is False # Fail Example
```

5. in and not in

```
note_TSC = [10, 4, 9, 9]
assert 10 in note_TSC || assert 5 not in note_TSC # Success Examples
assert 10 not in note_TSC || assert 742342342342 in note_TSC # Fail Examples
```

6. Greater than or less than

```
assert 5 > 4 # Success Example
assert 5 > 7 # Fail Example
```

7. any() | all()

```
note_TSC = [10, 4, 9, 8, 7]
trecut_TSC = [True, False, True, True, True]

assert any(note_TSC) == True # Success Example => cel putin un element din lista este diferit de 0.
assert any(trecut_TSC) == True # Success Example => cel putin un element din lista este True.

assert all(note_TSC) # Success Example => toate elementele din lista sunt diferite de 0.
assert all(trecut_TSC) # Fail Example => nu toate elementele din lista sunt True.
```

8. Exception

```
def function():
    raise SystemExit(1)


def test_mytest():
    with pytest.raises(SystemExit):
        function()
```

Marks

Marks pot fi folosite pentru aplicarea meta datelor la funcțiile de testare. Markerii sunt utilizați pentru a seta diverse caracteristici/atribute pentru a testa anumite funcționalități.

Markerii sunt declarați folosind următoarea sintaxă:

```
@pytest.mark.<markername>
```

Acest lucru ne permite gruparea testelor si rularea lor in anumite batch-uri

```
groceries = ["Potatoes", "Onions", "Flowers"]

@pytest.mark.veggies
def test_potatoes_in_list():
   assert 'Potatoes' in groceries

@pytest.mark.veggies
def test_onions_in_list():
   assert 'Onions' in groceries

@pytest.mark.others
def test_flowers_in_list():
   assert 'Flowers' in groceries
```

Pentru a rula o suită de deste marcate cu un anumit marker, se folosește comanda:

```
$ pytest -m <markername> -v
```

Pytest oferă o serie de marks deja implementate pentru a facilita testarea.

pytest.mark.skip

```
# https://docs.pytest.org/en/7.2.x/reference/reference.html#pytest-mark-skip

@pytest.mark.skip(reason="I have no idea how to test this")
def test_the_great_unknown():
    ...
```

pytest.mark.skipif

```
# https://docs.pytest.org/en/7.2.x/reference/reference.html#pytest-mark-skipif

@pytest.mark.skipif(sys.platform == "Linux", reason="Tests for Windows only!")
def test_function():
    ...
```

pytest.mark.xfail - atunci când ne așteptăm ca un test să întoarcă fail. xfail marchează testul ca fiind de așteptat să eșueze.

```
# https://docs.pytest.org/en/7.2.x/reference/reference.html#pytest-mark-xfail

def add_two_ints(a, b):
    return a + b

@pytest.mark.xfail(reason="Missing Arguments")
def test_add_two_ints_xfail_missing_arg() -> None:
    value = add_two_ints(10)  # Missing Argument - FAIL
    assert value == 10

...

@pytest.mark.xfail(sys.platform == "win32", reason="bug with the checkout", raises=RuntimeError)
def test_function():
    ...
```

Găsiți mai multe exemple de test skipping în documentația oficială: https://docs.pytest.org/en/7.1.x/how-to/skipping.html [https://docs.pytest.org/en/7.1.x/how-to/skipping.html]

pytest.mark.parametrize - permite definirea mai multor seturi de argumente pentru funcția sau clasa de testare. Așadar, parametrizarea unui test se face pentru a rula testul pe mai multe seturi de intrări.

```
@pytest.mark.parametrize("number, output", [(1, 2), (2, 3), (3, 6), (4, 5)])
def test_incrementation(number, output):
   assert number + 1 == output

@pytest.mark.parametrize("a, b, expected", [(1, 2, 4), (3, 4, 9)])
def test_i_know_how_to_count(a, b, expected):
    assert 2 * a + b == expected
```

Parametrizare unei ***clase*** se poate realiza astfel:

```
import pytest

@pytest.mark.parametrize("number, expected", [(1, 2), (3, 4)])
class TestClass:
    def test_increment(self, number, expected):
        assert number + 1 == expected

    def test_increment_with_extra_steps(self, number, expected):
        assert (number * 1) + 1 == expected
```

Dacă dorim parametrizarea tuturor testelor dintr-un modul, atunci putem realiza acest lucru prin initializarea **pytestmark** ca variabilă globală

```
import pytest

pytestmark = pytest.mark.parametrize("n,expected", [(1, 2), (3, 4)])

class TestClass:
    def test_increment(self, n, expected):
        assert n + 1 == expected

    def test_increment_with_extra_steps(self, n, expected):
        assert (n * 1) + 1 == expected
```

Fixtures - Introducere

Fixtures sunt **funcții** atașate la teste **care rulează înainte ca funcția de testare să fie executată**. Fixtures sunt folosite pentru a furniza unele date la teste, cum ar fi conexiunile la baze de date, URL-urile de testat sau un fel de date de intrare. Prin urmare, în loc să rulăm același cod pentru fiecare test, putem atașa funcția de fixture la teste și va returna datele testului înainte de a executa fiecare test.

Fixtures sunt un set de resurse care trebuiesc configurate înainte și curățate odată ce execuția testului este finalizată

Funcția pytest fixture este apelată automat de framework-ul pytest atunci când numele argumentului și numele fixture-ului sunt același.

O funcție este marcată ca element fixture folosind:

```
@pytest.fixture
```

O funcție de testare poate folosi un fixture prin menționarea numelui fixture-ului ca parametru de intrare.

```
@pytest.fixture
def fixture_function():
   return "TSC is love, TSC is life"


def test_fixture(fixture_function):
    assert fixture_function == "TSC is love, TSC is life"
```

Built-in fixtures: https://docs.pytest.org/en/7.2.x/reference/fixtures.html#fixture [https://docs.pytest.org/en/7.2.x/reference/fixtures.html#fixture]

Fixtures care sunt **autouse** se aplică fiecărui test care le-ar putea referi, fiind **executate înaintea tuturor**.

Dacă fixture-ul A este autouse și fixture-ul B nu este, dar fixture-ul A solicită fixture-ul B, atunci fixture-ul B va fi efectiv și un fixture autouse, dar numai pentru testele cărora li se aplică A.

Astfel, daca dorim executarea unei anumite logici înainte unui test, putem să realizăm acest lucru folosind autouse și scope. Scope marchează modul în care fixture-urile pot să fie "împărțite" (shared). Scope poate să fie: function, class, module, package or session.

```
@pytest.fixture(scope="session", autouse=True)
def do_something_before_everyone():
    # do something ahead of all the tests
    ...
```

Dacă dorim a avea o logică specifică înainte(set-up) și după(tear-down) fiecare test:

```
import pytest

@pytest.fixture(autouse=True)
def run_before_and_after_each_test():
    # Setup: logic that happens before the test

    yield # this is where the testing happens

    # Teardown: logic that happens after the test
```

Requesting & Finalization & Autouse & Scope for fixtures & Mai multe moduri de utilizare fixtures: https://docs.pytest.org/en/latest/how-to/fixtures.html [https://docs.pytest.org/en/latest/how-to/fixtures.html]

Trimiterea unui obiect catre test se poate realiza folosind fixture

```
import pytest

class Student:
  def __init__(self, name, age, scholarship):
    self.name = name
    self.age = age
    self.scholarship = scholarship

@pytest.fixture
def student():
    student = Student("Mr. Perfect", 20, True)

    return student

def test_student_creation(student):
    assert student.name == "Mr. Perfect"

    assert student.age == 20

    assert student.scholarship is True

    assert isinstance(student, Student)
```

Exerciții

https://github.com/Ingineria-Calculatoarelor-ACS-UPB/unit-testing-1-lab [https://github.com/Ingineria-Calculatoarelor-ACS-UPB/unit-testing-1-lab]

tsc/laboratoare/laborator-05.txt · Last modified: 2024/03/29 12:42 by dragos.sandulescu97